Development Plan: Trace-Driven Bug Reproduction and Patching

Companion to the project specification v1.0. Prepared November 1, 2026. Assumes about 12 weeks at 10 to 15 hours per week; scale the schedule to your real deadline.

1. How to Use This Plan

The specification says **what** the project is. This plan says **how and in what order** to build it, and how to know each piece is good enough before moving on.

Working rules:

- **Every task has acceptance criteria.** A task is done only when its criteria are met, not when the code is written.
- **Every week ends with something that runs.** Even if small. Never go two weeks without an end-to-end path working.
- **Track work in GitHub.** One issue per task, labels per workstream, and one GitHub milestone per phase. Close issues only when the acceptance criteria are met.
- **Spec changes follow the change rule** (spec section 15): logged amendment, and only for a harness bug or discovered infeasibility.
- **Write a weekly status note** (template in section 9). It keeps you honest and doubles as material for your professor and your write-up.

2. Quality Bar

These practices are what separate a portfolio-grade project from a demo:

- **The harness is tested more than anything else.** If the harness is wrong, every result is wrong. Unit, integration, and known-answer tests come before experiments.
- **Measure before optimizing.** Get one bug working end to end before generalizing.
- **Log everything, edit nothing.** Raw logs are immutable. Analysis reads only logs, so every table can be regenerated.
- **Small, reviewed changes.** Work on short-lived branches, open a pull request to yourself, read the diff before merging, and keep main green.
- **Honest reporting.** Failures and limitations are part of the deliverable, not something to hide.

Definition of done for any task: code merged to main; tests added and passing in CI; documentation updated if behavior changed; a one-line entry in CHANGELOG.md.

3. Workstreams

ID	Workstream	Scope
WS1	Dataset	Screening, preflight, manifests, payloads
WS2	Sandbox and harness	Image builder, container runner, patch validator
WS3	Agent	LLM client, prompts, Pipelines A and B, feedback loops
WS4	Evaluation	Oracles, fresh-container scoring, taxonomy labeling
WS5	Analysis	Statistics, figures, manual audit
WS6	Live demo	Webhook receiver, draft-PR publisher
WS7	Docs and portfolio	README, docs, write-up, video
WS8	Business validation	Interviews and notes (light, parallel)

4. Phase 0: Setup (Week 1)

Goal: a clean, professional repository and the first manual look at a real bug.

Tasks:

1. Create the GitHub repository with the layout from spec section 11, an MIT license, and the spec committed and tagged v1.0.
2. Set up the development environment: a modern Python for the harness itself, `uv` (or pip-tools) with a lock file, `ruff`, `pytest`, `pre-commit`, and optionally `mypy`.
3. Add GitHub Actions running ruff and pytest on every push.
4. Install Docker and confirm `docker run --net=none python:3.11-slim python -c 1` works.
5. Add a Makefile with placeholder targets: `build-images`, `validate-gold`, `run-benchmark`, `analyze`, `test`.
6. Create `.env.example` for API keys, add `.env` to `.gitignore`, and add a secret scanner to pre-commit (such as gitleaks).
7. Build a **toy library fixture** in `tests/fixtures/tiny_lib`: a 50-line package with a planted crash bug, a buggy version, a fixed version, and a regression test. All fast tests run against this, so you never need a real repository to test the harness.
8. Start the screening log with 10 candidates from click, requests, and flask.

9. Take one real candidate through the gold-patch preflight **by hand** with plain git and docker commands. Write down every problem you hit.

Acceptance criteria: CI is green on an empty test; the toy fixture exists with a failing and a passing version; one real bug has been manually validated or rejected with a written reason.

5. Phase 1: Harness and Dev Bugs (Weeks 2 to 3)

Goal: a trustworthy sandbox and evaluator, proven on the toy fixture and 5 real dev bugs.

5.1 Image builder (WS2)

- Script that, given a manifest, checks out the parent commit, strips `.git`, writes a Dockerfile with the pinned Python version and dependencies, and builds `bug_env:{id}`.
- **Acceptance:** builds the toy fixture and one real bug from a single command; a rebuild produces the same installed package versions.

5.2 Container runner (WS2)

- A function that runs a command in a locked-down container (all flags from spec section 7.2) against a throwaway writable copy of the repo, enforces a timeout, and returns exit code, output, and duration.
- Use pytest's `--junitxml` output for results instead of parsing console text.
- Write a small **harness-owned pytest plugin**, mounted read-only, that records the exception type and innermost frame of each failure to a JSON file. The repro gate and oracle checks read this file.
- **Acceptance:** sandbox meta-tests pass. Network access fails; writing outside the work directory fails; a fork bomb hits the PID limit; an infinite loop is killed at the timeout; no host environment variables are visible.

5.3 Patch validator (WS2)

- Host-side checks from spec section 7.1, applied to the final diff.
- **Acceptance:** one unit test per rule, each with a passing and a rejected example (path traversal, symlink, binary, test edit, new `conftest.py`, `sitecustomize.py`, workflow edit, oversized diff).

5.4 Gold preflight and manifests (WS1)

- Automate the four preflight checks from spec section 5.4 and write the manifest file per bug.
- Payload builder: run the gold oracle test on the buggy commit with locals captured and format a Sentry-like JSON payload.
- **Acceptance:** 5 real dev bugs pass preflight with manifests and payloads; screening log records every rejected candidate.

5.5 Known-answer and red-team tests (WS4, WS2)

- For every dev bug: the gold patch is scored correct; an empty patch is scored incorrect; a patch that edits tests or adds a skipping `conftest.py` is rejected.
- Red-team test: a payload whose exception message contains injection instructions, plus a malicious patch attempting network access, secret reads, and a workflow edit. Every attempt is contained or rejected.

Milestone M1 (end of week 3): harness validated on toy fixture and 5 dev bugs; all known-answer and red-team tests pass in CI (sandbox tests may run locally if CI lacks Docker).

6. Phase 2: The Agent Stage (Weeks 4 to 5)

Goal: Pipelines A and B running end to end on the dev bugs, with measured cost.

Build **Pipeline B first**. It is the simpler subset (context, patch, feedback), and Pipeline A reuses all of it plus the repro stage.

6.1 LLM client (WS3)

- One interface with a provider adapter per vendor (`complete(messages, settings) -> response, usage`).
- **Separate API failures from model attempts.** Rate limits, timeouts, and server errors are retried with exponential backoff and never count as one of the agent's attempts. If retries are exhausted, mark the run as an infrastructure failure and rerun it later; it is never a model failure.
- Log every call: timestamp, model identifier, prompt version, request hash, full request, full response, token usage, latency, cost.
- Development cache: identical requests return the cached response, so debugging later stages does not re-spend money. **Disable the cache for final runs**, since trials must be independent.
- Fixed settings per experiment: model identifier, temperature, and maximum output tokens, all recorded in the run config.

- **Acceptance:** unit tests with a fake provider; one real call per vendor logged correctly.

6.2 Context assembler (WS3)

- Collect the files of trace frames inside the repository, plus 1-hop imports using Python's `ast`. Exclude tests and anything outside the package.
- Present files with their paths and line numbers. Apply the token cap with a documented truncation rule (for example, keep trace-frame files whole and trim imported files to their top-level definitions first).
- **Acceptance:** deterministic output (same input gives the same context); unit tests on the toy fixture; context size logged per bug.

6.3 Prompts and output contracts (WS3)

- Store prompts as versioned files: `prompts/repro_v1.md`, `prompts/patch_v1.md`, and a shared system prompt. Untrusted payload content always goes inside clearly delimited data sections.
- **Repro output contract:** exactly one Python test file in a single fenced code block. The harness extracts it and places it in its own repro directory. It is never part of the patch, so the deny-list on test edits still applies to patches.
- **Patch output contract:** use **search-and-replace edit blocks** (an exact snippet of existing code and its replacement, per file) rather than asking the model for a raw unified diff. Models often produce unified diffs with wrong line numbers that fail to apply. The harness applies the edits, generates the real diff, and runs the validator on that diff.
- Malformed output (no code block, a snippet that does not match the file) counts as a failed attempt and is labeled Syntax or Import Error or Scope Violation per the taxonomy.
- **Acceptance:** parser unit tests with well-formed and malformed responses.

6.4 Pipeline B: direct patch (WS3)

1. Send system prompt, payload, and context; ask for a patch.
2. Validate and apply the patch; run the visible baseline suite in the sandbox.
3. If tests regress and attempts remain, send back the names of failing tests and the last part of their output (for example, the final 60 lines), then ask for a revised patch.
4. Submit the final patch for evaluation.

6.5 Pipeline A: repro-first (WS3)

1. Ask for a repro test (attempt 1 of 2).

2. Run it on the buggy code. **Agent-time gate:** it must fail with the target exception type from the target frame, as read from the harness plugin output.
3. If the gate fails, send back what happened (passed, or failed with exception X at frame Y instead of the target), and retry once.
4. If no valid repro after 2 attempts, stop and record No-Repro or Invalid Repro.
5. Ask for a patch, including the valid repro test and its failure output.
6. Validate, apply, run the repro test plus the baseline suite. If the repro still fails or tests regress, send back the results and retry once.
7. Submit the final patch for evaluation.

Feedback rule for both pipelines: the agent only ever sees results from the buggy code, its own repro test, and the visible baseline tests. It never sees `C_fix`, the oracle tests, or evaluation results.

6.6 Budget accounting (WS3)

- Count tokens from each API response's usage fields. Both pipelines share the same per-run token cap; when it is reached, the run stops and the current best patch (if any) is submitted.
- Run both pipelines on the 5 dev bugs, measure the average tokens and cost per run, then set the final cap and a global spending limit for the full experiment.

Milestone M2 (end of week 5): both pipelines run end to end on all dev bugs from one command; per-run cost measured; caps set.

7. Phase 3: Evaluation and Dataset Scale-Up (Weeks 5 to 8)

7.1 Evaluator (WS4)

- Fresh container per evaluation: copy the oracle tests in from `C_fix`, re-apply the validated diff from the host, run oracle tests plus the stable baseline set.
- Score the repro oracle for Pipeline A (all three conditions in spec section 7.4).
- **Automatic taxonomy labels** for every category except State Deficit, following the precedence in spec section 8. Fault Localization Miss is computed by comparing `gold_files` with the context file list.
- **Acceptance:** known-answer tests from Phase 1 pass through the full evaluator; each taxonomy label has a test case on the toy fixture.

7.2 Dataset scale-up (WS1)

- Screen candidates in batches of 10 to 20 per week. Record every decision in the screening log.
- Aim for 30 validated test-set bugs, disjoint from the dev set.
- **Trigger date:** if fewer than 20 bugs are validated by the end of week 7, freeze at the number you have (minimum 20) and stop screening. Do not let curation eat the experiment.

7.3 Dry run (WS3, WS4)

- Full end-to-end run on the dev set with both pipelines and both models, cache disabled, exactly as the final run will work.
- Fix any harness bugs found (logged as amendments if they change the spec).

Milestone M3 (end of week 8): the freeze. Tag the repository with the frozen dataset manifests, prompt versions, harness version, model identifiers, and the analysis plan. After this tag, nothing changes before the final run except harness bug fixes.

8. Phases 4 to 6: Run, Analyze, Ship (Weeks 9 to 12)

8.1 Final runs (week 9, WS3)

- Run on a disposable Linux VM with no personal secrets. Write a run ID that includes the harness commit hash, prompt version, and model identifier.
- Store results as `runs/{run_id}/{bug_id}/{pipeline}/{model}/trial_{n}/` containing the raw LLM logs, generated files, sandbox outputs, and verdict JSON.
- Make the runner **resumable**: if it crashes, it skips completed trials instead of restarting.
- Rerun infrastructure failures only; never rerun a trial because you dislike its result.

8.2 Analysis (week 10, WS5)

- Analysis scripts read only the stored logs and produce all tables and figures (spec sections 8, 9, and 12).
- Manual audit: review every correct patch and about 25% of incorrect ones; record findings in `docs/AUDIT.md` with the estimated oracle false-negative rate.
- Label State Deficit failures blind to pipeline and model where possible.
- **Acceptance:** `make analyze` regenerates every table and figure from a clean clone without API keys.

8.3 Live demo (week 11, WS6)

- FastAPI webhook receiver that converts a Sentry-style payload into the same internal format as a manifest.
- GitHub publisher that opens a **draft** PR on your own fork, containing the repro test, the patch, and a short root-cause summary. The token is scoped to that fork only.
- Record a 2 to 3 minute demo video.

8.4 Documentation and write-up (weeks 11 to 12, WS7)

- README with the headline result, results table, architecture diagram, and a quickstart that works in under 5 minutes from cached logs.
- `docs/`: ARCHITECTURE.md, METHODOLOGY.md, LIMITATIONS.md, SECURITY.md, AUDIT.md.
- Four case studies (two successes, two failures).
- Write-up of 1,500 to 2,500 words. Ask a classmate to read the README cold and tell you where they got lost.

Milestone M4 (end of week 10): results and figures final. **Milestone M5 (end of week 12):** public release.

9. Schedule, Milestones, and Status Notes

Week	Main work	Checkpoint
1	Repo, tooling, toy fixture, first manual preflight, screening starts	
2	Image builder, container runner, patch validator	
3	Preflight automation, payload builder, known-answer and red-team tests, 5 dev bugs	M1 harness validated
4	LLM client, context assembler, prompts, Pipeline B	
5	Pipeline A, feedback loops, cost measurement, caps	M2 pipelines end to end
6	Evaluator, taxonomy labels, screening scale-up, start interviews	
7	Screening, dry run on dev set	Trigger: at least 20 bugs?
8	Final fixes, freeze tag	M3 freeze

Week	Main work	Checkpoint
9	Full runs on VM	
10	Analysis, audit, labeling, figures	M4 results final
11	Live demo, documentation	
12	Write-up, video, final checklist, buffer	M5 release

Weekly status note template (keep in `docs/status/`):

- Done this week (with links to closed issues)
- Planned for next week
- Blockers and risks
- Numbers: bugs validated, tests passing, money spent so far
- Decisions made or amendments logged

Share milestone notes with your professor at M1, M3, and M5 at minimum.

10. Testing Strategy

Layer	What it covers	Where it runs
Unit	Validator rules, trace parsing, context assembler, output parsers, taxonomy logic	CI, every push
Integration	Sandbox runner and evaluator against the toy fixture	CI if Docker is available, otherwise locally before each merge
Known-answer	Gold patch correct, empty patch incorrect, tampering rejected, per dev bug	Locally before each milestone
Red-team	Injection payload and malicious patch are contained	CI or locally, before each milestone
Regression	One new test for every harness bug you find	CI

Rule: **every harness bug becomes a test before it is fixed.**

11. Risks and Contingencies

Risk	Early warning	Response
Low dataset yield	Fewer than 10 validated bugs by week 5	Add backup repositories; relax the date preference, never the oracle checks
Environment build pain	A single bug takes more than about 3 hours to build	Drop it and log the reason
Cost overrun	Dry-run cost extrapolates above budget	Reduce trials or models via the scope-cut ladder (spec section 13)
Harness bug after freeze	Known-answer test fails	Fix, log an amendment, rerun affected trials only
Model or API change	Deprecation notice	Pinned identifiers and cached logs; note in limitations
Schedule slip	A milestone misses by more than a week	Apply the scope-cut ladder immediately, not at the end

12. Business Validation Track (Weeks 6 to 10)

- Weeks 6 to 7: write a 10-question interview script focused on how teams handle production bugs today and where time goes. Do not pitch.
- Weeks 7 to 10: hold about 10 conversations; keep notes in `docs/interviews/` (private, not in the public repo).
- Week 10: write a one-page summary and apply the kill criterion from spec section 14.

Cap this at about 1 to 2 hours per week so it never delays the core project.

13. Final Quality Checklist (Before Release)

- Fresh clone, follow the README quickstart exactly: it works and `make analyze` reproduces every figure.
- All tests pass; CI is green on main.
- Secret scan of the full git history is clean; no API keys or tokens anywhere.
- Results table shows raw counts, percentages, and confidence intervals.
- LIMITATIONS.md covers contamination, synthetic payloads, small N, oracle false negatives, Python-only scope, and Docker isolation limits.

- SECURITY.md states that delimiters are not a security boundary and describes the tested containment.
- Case studies, figures, demo video, and write-up are complete and proofread.
- Repository has a clear description, topics, license, and a pinned release tag.
- Resume bullet and interview talking points updated with real numbers.

14. This Week

1. Create the repository, commit the spec, tag v1.0, and set up the GitHub project board with the Phase 0 and Phase 1 issues from this plan.
2. Set up tooling and CI, and build the toy library fixture.
3. Screen the first 10 candidates and take one through preflight by hand.